

```

long N_iterations=1000000;
float avgTimeSysCall, avgTimeFuncCall; struct timeval t1,
t2;
res=gettimeofday(&t1,NULL);
assert(res==0);
for (i=0;i<N_iterations; i++)
{
j=getpid();
}
res=gettimeofday(&t2,NULL);
assert(res==0);
avgTimeSysCall = (nanosec(t2) - nanosec(t1))/(N_
iterations*1.0);
res=gettimeofday(&t1,NULL);
assert(res==0);
for (i=0;i<N_iterations; i++)
{
j=foo();
}
res=gettimeofday(&t2,NULL);
assert(res==0);
avgTimeFuncCall = (nanosec(t2) - nanosec(t1))/(N_
iterations*1.0);
printf("Average time for System call getpid :
%f\n", avgTimeSysCall);
printf("Average time for Function call :
%f\n", avgTimeFuncCall);
}

```

```

jishnu@jishnu-VirtualBox:~/Desktop$ gcc jishnu.c
jishnu@jishnu-VirtualBox:~/Desktop$ ./a.out
Average time for System call getpid : 452.549988
Average time for Function call : 1.685000
jishnu@jishnu-VirtualBox:~/Desktop$

```

Figure 1: Output of time difference between a function call and system call

In this code, we are taking very simple system and function calls and running them multiple times, calculating averages of all the calls to get an accurate value. We are measuring the time in nanoseconds, as the durations are that short!!

You will see in the output, as shown in Figure 1, that the system call takes approximately ~452 nanoseconds whereas the function call requires only ~1.6 nanoseconds, which is way less!! This proves that context switching is a huge overhead during function calls, emphasising the need for caution.

This was very interesting, wasn't it? Linux is fascinating and the fact that we can experiment and learn on it adds to its allure. So, never stop exploring and learning — the deeper you go, the more fun it is!! **END** 🐧

 By: Jishnu Saurav Mittapalli

The author is a full-stack web developer, interested in modern technologies like artificial intelligence and machine learning. He is currently working in the field of natural language processing.

OSFY Magazine Attractions During 2024-25

Month	Theme
March 2024	Gen AI, Deep learning and Machine Learning
April 2024	Open Source Programming (Languages and tools)
May 2024	Blockchain and Open Source
June 2024	Security, Network Management and Monitoring
July 2024	All About Data Management (Database management, Optimisation, Big Data, Data analysis, etc)
August 2024	Mobile/Web App Development, Optimisation and Security
September 2024	DevOps Special
October 2024	Cloud Special: Everything from management to implementation
November 2024	Containers and Managing Containers
December 2024	Open Source and Quantum Computing
January 2025	Open Source and IoT and Edge
February 2025	Open Source on Windows and Best in the world of Open Source (Tools and Services)